

NECCDC 2026 — Runbook 4 of 4

DC02 Full Hardening Runbook

IP: 10.100.6.64 — oceancrest.local branch domain controller.

DC02 is different from DC01 because you have already been on it. This runbook covers what may have been missed on Day 1, what Red Team will re-attempt on Day 2, and every hardening step with full PowerShell commands and what-if scenarios.

DC02 vs DC01 — Key Differences

You spent Day 1 on DC02. This means two things: you have already applied some hardening, but Red Team has also been watching your activity and learning your patterns. Day 2 on DC02 is about verifying what you did held, catching anything you missed, and handling the cross-domain trust relationship correctly now that DC01 may be accessible.

Aspect	DC01 (Unknown)	DC02 (Your Machine)
Hardening status	None — assume fully compromised	Partial — verify Day 1 work held
Persistence risk	Definitely present	May have been replanted overnight
Your access	Need to find and connect	Already have RDP credentials
Domain role	ad.chefops.local primary	ad.oceancrest.local branch
Trust direction	Primary side of trust	Branch side — trust built here
Replication	Source for chefops objects	Replicates from DC01

■ Even though you hardened DC02 on Day 1, Red Team worked overnight. Treat the first 10 minutes of Day 2 as a re-verification of everything you did yesterday before assuming it is still clean.

Phase 0 — Day 2 Morning Re-Verification (First 10 Minutes)

Run all of these before touching anything else. Screenshot every output. If anything has changed from what you left it, you have an active Red Team incident and need to start an IR report immediately.

0.1 — Persistence Re-Check

```
# Scheduled tasks outside Microsoft namespace
Get-ScheduledTask |
  Where-Object {$_.TaskPath -notlike '\Microsoft\*'} |
  Select TaskName, TaskPath, State

# WMI subscriptions - should be empty
Get-WMIObject -Namespace root\subscription -Class __EventFilter
Get-WMIObject -Namespace root\subscription -Class __EventConsumer

# Registry autorun keys
$keys = @(
  'HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Run',
  'HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\RunOnce',
  'HKLM:\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon'
)
foreach ($k in $keys) {
  Write-Host "`n=== $k ===" -ForegroundColor Yellow
  Get-ItemProperty -Path $k -ErrorAction SilentlyContinue
}
```

0.2 — Privileged Group Membership Re-Check

```
# Domain Admins - any additions overnight?
Get-ADGroupMember 'Domain Admins' | Select Name, SamAccountName

# Enterprise Admins
Get-ADGroupMember 'Enterprise Admins' | Select Name, SamAccountName

# New accounts created since yesterday
Get-ADUser -Filter * -Properties Created |
  Where-Object {$_.Created -gt (Get-Date).AddHours(-12)} |
  Select Name, SamAccountName, Created, Enabled
```

0.3 — Verify Firewall Rules Still Intact

```
# All profiles still enabled?
Get-NetFirewallProfile | Select Name, Enabled, DefaultInboundAction

# Any new inbound allow rules added overnight?
Get-NetFirewallRule -Direction Inbound -Action Allow -Enabled True |
  Select DisplayName, Profile | Sort DisplayName

# Specifically look for rules Red Team may have added
Get-NetFirewallRule |
  Where-Object {$_.Description -eq $null -and $_.Action -eq 'Allow'} |
  Select DisplayName, Direction, Profile
```

What If: New firewall allow rule appeared that you did not add

■ Red Team added a backdoor firewall rule. Remove it immediately and document for IR:

```
# Remove the rule
Remove-NetFirewallRule -DisplayName 'SuspiciousRuleName'

# Log the incident
Write-Host "Removed rogue rule at $(Get-Date)" | Out-File C:\incident_log.txt -Append
```

0.4 — Verify ms-DS-MachineAccountQuota Still 0

```
Get-ADObject -Identity ((Get-ADDomain).DistinguishedName) `
  -Properties ms-DS-MachineAccountQuota

# If it reverted to 10, set it back
Set-ADDomain -Identity (Get-ADDomain).DNSRoot `
  -Replace @{'ms-DS-MachineAccountQuota'='0'}
```

0.5 — Check Replication Health With DC01

```
repadmin /showrepl
repadmin /replsummary

# Check if DC01 is now reachable
Get-ADDomainController -Filter * | Select Name, IPv4Address, Domain
```

Phase 1 — Items Commonly Missed on Day 1

Based on competition feedback patterns, these are the areas most likely to have been skipped or only partially completed under time pressure on Day 1. Work through each one.

1.1 — KRBTGT Password (Golden Ticket Prevention)

```
# Check when KRBTGT password was last set
Get-ADUser krbtgt -Properties PasswordLastSet |
    Select Name, PasswordLastSet

# If not reset yesterday, reset it TWICE with 10 min gap
# Reset 1
Set-ADAccountPassword -Identity krbtgt `
    -NewPassword (ConvertTo-SecureString "$(New-Guid)$(New-Guid)" -AsPlainText -Force)

# WAIT 10 MINUTES - then reset again
# Reset 2
Set-ADAccountPassword -Identity krbtgt `
    -NewPassword (ConvertTo-SecureString "$(New-Guid)$(New-Guid)" -AsPlainText -Force)
```

■ Two resets 10 minutes apart are required to fully invalidate any existing Golden Tickets. One reset is not enough. Coordinate with team — brief auth disruption will occur.

1.2 — GPO Evidence You May Have Missed

The quals feedback specifically said teams configured GPOs but did not provide proof of enforcement. Run these and screenshot the output — this is scored evidence.

```
# Verify PowerShell logging GPO is actually applied
gpreresult /r /scope computer

# Check on multiple machines remotely
$machines = @('DC02', '', '')
foreach ($m in $machines) {
    Write-Host "`n=== $m ==="
    Invoke-Command -ComputerName $m -ScriptBlock {gpreresult /r /scope computer}
}

# Force refresh across domain
Invoke-GPUUpdate -Computer DC02 -Force

# Verify Script Block Logging is actually writing events
Get-WinEvent -LogName 'Microsoft-Windows-PowerShell/Operational' `
    -MaxEvents 5 -ErrorAction SilentlyContinue | Select TimeCreated, Message
```

What If: gpreresult shows GPO not applied

```
# Check GPO link status
Get-GPInheritance -Target (Get-ADDomain).DistinguishedName

# Check if GPO is enabled
Get-GPO -All | Select DisplayName, GpoStatus

# Re-link GPO if missing
New-GPLink -Name 'YourGPOName' -Target (Get-ADDomain).DistinguishedName
```

```
# Force refresh
gpupdate /force
```

1.3 — Audit Policy Verification

```
# Full audit policy status
auditpol /get /category:*

# Set anything that shows No Auditing
auditpol /set /subcategory:'Logon' /success:enable /failure:enable
auditpol /set /subcategory:'Account Logon' /success:enable /failure:enable
auditpol /set /subcategory:'Account Management' /success:enable /failure:enable
auditpol /set /subcategory:'DS Access' /success:enable /failure:enable
auditpol /set /subcategory:'Policy Change' /success:enable /failure:enable
auditpol /set /subcategory:'Privilege Use' /success:enable /failure:enable
auditpol /set /subcategory:'Process Creation' /success:enable /failure:enable
```

1.4 — Kerberos Attack Surface

```
# AS-REP Roastable accounts
Get-ADUser -Filter {DoesNotRequirePreAuth -eq $true} `
  -Properties DoesNotRequirePreAuth | Select Name, SamAccountName

# Fix any found
Get-ADUser -Filter {DoesNotRequirePreAuth -eq $true} |
  Set-ADUser -DoesNotRequirePreAuth $false

# User accounts with SPNs (Kerberoastable)
Get-ADUser -Filter {ServicePrincipalName -ne '$null'} `
  -Properties ServicePrincipalName |
  Select Name, SamAccountName, ServicePrincipalName

# Force AES on service accounts
Get-ADUser -Filter {ServicePrincipalName -ne '$null'} |
  Set-ADUser -KerberosEncryptionType AES256,AES128
```

1.5 — AdminSDHolder and DCSync ACL Audit

```
# AdminSDHolder - persistent privilege backdoor
$acl = Get-ACL 'AD:\CN=AdminSDHolder,CN=System,DC=chefops,DC=local'
$acl.Access |
  Where-Object {
    $_.IdentityReference -notlike '*Domain Admins*' -and
    $_.IdentityReference -notlike '*Enterprise Admins*' -and
    $_.IdentityReference -notlike '*SYSTEM*' -and
    $_.IdentityReference -notlike '*Administrators*'
  } | Select IdentityReference, ActiveDirectoryRights

# DCSync rights - only DCs and Domain Admins should appear
$dn = (Get-ADDomain).DistinguishedName
$acl2 = Get-ACL "AD:\$dn"
$acl2.Access | Where-Object {
  $_.ObjectType -eq '1131f6aa-9c07-11d1-f79f-00c04fc2dcd2' -or
  $_.ObjectType -eq '1131f6ab-9c07-11d1-f79f-00c04fc2dcd2'
} | Select IdentityReference, ObjectType
```

Phase 2 — DC02-Specific Hardening

DC02 is the oceancrest branch DC. It serves the POS and Kiosk machines on the 10.100.6.x subnet. Its hardening needs to account for those client machines specifically.

2.1 — Harden the DC02 Firewall for Its Role

```
# Verify all profiles on
Get-NetFirewallProfile | Select Name, Enabled, DefaultInboundAction
Set-NetFirewallProfile -Profile Domain,Private,Public -Enabled True

# Allow only required inbound on Public profile
# These are the ports DC02 legitimately needs
$dcPorts = @(53,88,135,389,445,464,636,3268,3269,3389,5985,5986)
foreach ($port in $dcPorts) {
    New-NetFirewallRule -DisplayName "Allow DC Port $port" `
        -Direction Inbound -Protocol TCP -LocalPort $port `
        -Profile Domain -Action Allow -ErrorAction SilentlyContinue
}

# Block management ports on Public profile
New-NetFirewallRule -DisplayName 'Block RDP Public' `
    -Direction Inbound -Protocol TCP -LocalPort 3389 `
    -Profile Public -Action Block

New-NetFirewallRule -DisplayName 'Block WinRM Public' `
    -Direction Inbound -Protocol TCP -LocalPort 5985,5986 `
    -Profile Public -Action Block
```

2.2 — Restrict What Kiosks and POS Can Do to DC02

```
# Block LDAP enumeration from kiosk subnet
# Replace with actual kiosk subnet
New-NetFirewallRule -DisplayName 'Restrict LDAP from Kiosks' `
    -Direction Inbound -Protocol TCP -LocalPort 389 `
    -RemoteAddress /24 -Action Block

# Block Global Catalog from kiosks
New-NetFirewallRule -DisplayName 'Block GC from Kiosks' `
    -Direction Inbound -Protocol TCP -LocalPort 3268,3269 `
    -RemoteAddress /24 -Action Block

# Allow POS to DC on required ports only
New-NetFirewallRule -DisplayName 'Allow POS Auth to DC' `
    -Direction Inbound -Protocol TCP -LocalPort 88,389,445 `
    -RemoteAddress 10.100.6.37 -Action Allow
```

2.3 — Verify SMBv1 is Disabled

```
Get-SmbServerConfiguration | Select EnableSMB1Protocol

# Disable if still on
Set-SmbServerConfiguration -EnableSMB1Protocol $false -Force

# Also check SMB signing is enforced
Get-SmbServerConfiguration | Select RequireSecuritySignature, EnableSecuritySignature

# Enable SMB signing
```

```
Set-SmbServerConfiguration -RequireSecuritySignature $true -Force
```

2.4 — Disable Unnecessary Services on DC02

```
# Same as DC01 - these should have been done Day 1
# Re-verify each one

# Print Spooler
(Get-Service Spooler).StartType
# Should be Disabled - if not:
Stop-Service Spooler -Force
Set-Service Spooler -StartupType Disabled

# Remote Registry
(Get-Service RemoteRegistry).StartType
Stop-Service RemoteRegistry -Force -ErrorAction SilentlyContinue
Set-Service RemoteRegistry -StartupType Disabled

# Fax
(Get-Service Fax).StartType
Stop-Service Fax -Force -ErrorAction SilentlyContinue
Set-Service Fax -StartupType Disabled

# Verify PowerShell v2 disabled
Get-WindowsOptionalFeature -Online -FeatureName MicrosoftWindowsPowerShellV2Root
```

Phase 3 — Domain Trust Hardening

Once DC01 is found and a trust is established between chefops.local and oceancrest.local, these hardening steps must be applied immediately. An unhardened trust is a lateral movement highway.

3.1 — Verify Trust Status

```
# Check what trusts exist
Get-ADTrust -Filter * | Format-List *

# Key properties to check
Get-ADTrust -Filter * |
    Select Name, TrustType, TrustDirection, TrustAttributes,
        SIDFilteringQuarantined, SelectiveAuthentication
```

3.2 — Enable SID Filtering Immediately After Trust Creation

```
# SID filtering prevents cross-domain privilege escalation
# Run this from DC02 (oceancrest side)
netdom trust ad.oceancrest.local `
    /domain:ad.chefops.local `
    /quarantine:yes

# Verify it took effect
Get-ADTrust -Filter * | Select Name, SIDFilteringQuarantined
```

■ Without SID filtering, a compromised oceancrest account can forge SID history to claim Domain Admin rights in chefops.local. This is a complete forest takeover vector. Enable this before anything else.

3.3 — Enable Selective Authentication

```
# Selective auth means oceancrest users can only access
# chefops resources you explicitly grant them
netdom trust ad.oceancrest.local `
    /domain:ad.chefops.local `
    /SelectiveAuth:yes

# Verify
Get-ADTrust -Filter * | Select Name, SelectiveAuthentication
```

3.4 — Audit Cross-Domain Group Memberships

```
# Check if any oceancrest accounts are in chefops privileged groups
# Run this from DC02
Get-ADGroupMember 'Domain Admins' |
    Where-Object {$_.distinguishedName -like '*oceancrest*'}

# Check all privileged groups
$groups = @('Domain Admins', 'Enterprise Admins', 'Schema Admins',
    'Backup Operators', 'DNSAdmins')
foreach ($g in $groups) {
    $members = Get-ADGroupMember -Identity $g -ErrorAction SilentlyContinue
    $cross = $members | Where-Object {$_.distinguishedName -like '*oceancrest*'}
    if ($cross) {
        Write-Host "CROSS-DOMAIN MEMBER IN $g" -ForegroundColor Red
        $cross | Select Name, SamAccountName
    }
}
```

```
}
```

What If: Oceancrest account found in chefops privileged group

■ This is either Red Team persistence or a misconfiguration. Remove the account and document for IR immediately:

```
Remove-ADGroupMember -Identity 'Domain Admins' `
  -Members 'oceancrest\rogue_account' -Confirm:$false
```

Phase 4 — Account Hardening Specific to DC02

4.1 — Oceancrest User Account Audit

```
# All oceancrest domain users
Get-ADUser -Filter * -Properties Created, LastLogonDate, MemberOf |
  Select Name, SamAccountName, Enabled, Created, LastLogonDate |
  Sort Created -Descending

# Users with no logon history (potential planted accounts)
Get-ADUser -Filter * -Properties LastLogonDate |
  Where-Object {$_.LastLogonDate -eq $null -and $_.Enabled -eq $true} |
  Select Name, SamAccountName, Created

# Users with passwords set to never expire
Get-ADUser -Filter {PasswordNeverExpires -eq $true} `
  -Properties PasswordNeverExpires |
  Select Name, SamAccountName
```

4.2 — POS and Kiosk Service Accounts

```
# Find the service accounts used by POS and kiosks
Get-ADUser -Filter * -SearchBase 'OU=ChefOps_ServiceAccounts,DC=oceancrest,DC=local' `
  -ErrorAction SilentlyContinue | Select Name, SamAccountName

# Check their group memberships - should be minimal
Get-ADUser -Filter {Name -like '*pos*' -or Name -like '*kiosk*'} `
  -Properties MemberOf |
  Select Name, @{N='Groups';E={$_.MemberOf -join ', '}}

# Verify POS service account password is strong
# Reset it if it has not been changed today
Set-ADAccountPassword -Identity 'pos-svc' `
  -NewPassword (ConvertTo-SecureString 'P0sS3rvic3!2026#' -AsPlainText -Force)
```

4.3 — Local Accounts on DC02 Itself

```
# All local accounts on DC02
Get-LocalUser | Select Name, Enabled, LastLogon, PasswordLastSet

# Local administrators group
Get-LocalGroupMember -Group 'Administrators'

# Disable any local account you do not recognize
Disable-LocalUser -Name 'UnknownAccount'

# Change local Administrator password
net user Administrator 'N3wStr0ng!LocalP@ss2026'
```

4.4 — Password Policy for Oceancrest Domain

```
# Set strong password policy
Set-ADDefaultDomainPasswordPolicy `
  -Identity (Get-ADDomain).DNSRoot `
  -LockoutThreshold 5 `
  -LockoutDuration '00:15:00' `
  -LockoutObservationWindow '00:15:00' `
  -MinPasswordLength 14 `
  -PasswordHistoryCount 12 `
```

```
-ComplexityEnabled $true
```

```
# Verify
```

```
Get-ADDefaultDomainPasswordPolicy
```

Phase 5 — Replication Health and Monitoring

DC02 replicates from DC01. If Red Team breaks replication, DC02 becomes out of sync — authentication starts failing for oceancrest users and service checks begin to fail. Monitor this actively.

5.1 — Check Replication Health

```
# Full replication status
repadmin /showrepl

# Summary view
repadmin /replsummary

# Check for replication failures
repadmin /showrepl * /errorsonly

# Check replication queue
repadmin /queue
```

What If: Replication shows failures

```
# Force a manual sync
repadmin /syncall /AdeP

# Sync with specific DC
repadmin /replicate DC02 DC01 dc=oceancrest,dc=local

# Check for USN rollback (serious replication corruption)
repadmin /showutdvec DC02

# Check netlogon service
Get-Service Netlogon | Select Status, StartType
Restart-Service Netlogon
```

5.2 — FSMO Role Verification

```
# Check where FSMO roles sit
Get-ADDomain | Select PDCEmulator, RIDMaster, InfrastructureMaster
Get-ADForest | Select SchemaMaster, DomainNamingMaster

# Netdom view
netdom query fsmo
```

■ If the PDC Emulator role is on DC01 and DC01 is compromised or unreachable, time synchronization and password changes across the domain may fail. Seize the role to DC02 only if DC01 is confirmed down:

```
# ONLY if DC01 is permanently unreachable
# Seize PDC Emulator to DC02
ntdsutil
# At ntdsutil: roles
# At fsmo maintenance: connections
# At server connections: connect to server DC02
# At server connections: quit
# At fsmo maintenance: seize pdc
# Confirm when prompted
```

5.3 — Active Directory Database Health

```
# Check AD database integrity
ntdsutil 'activate instance ntds' 'files' 'integrity' quit quit

# Check event log for AD-specific errors
Get-WinEvent -LogName 'Directory Service' `
  -MaxEvents 20 -ErrorAction SilentlyContinue |
  Where-Object {$_.LevelDisplayName -eq 'Error' -or
    $_.LevelDisplayName -eq 'Warning'} |
  Select TimeCreated, Message | Format-List
```

Phase 6 — Active Monitoring During Competition

Run these checks on a rotation. Assign one team member to monitor DC02 while others handle injects. Every 30 minutes minimum.

6.1 — Event ID Monitoring Script

```
# Pull critical events from last 30 minutes
function Check-DCEvents {
    $events = Get-WinEvent -FilterHashtable @{
        LogName     = 'Security'
        StartTime   = (Get-Date).AddMinutes(-30)
        Id          = @(4720,4722,4723,4724,4725,4726,
                        4728,4732,4740,4756,4625,4648,4672)
    } -ErrorAction SilentlyContinue

    if ($events) {
        Write-Host "ALERT: Security events detected!" -ForegroundColor Red
        $events | Select TimeCreated, Id, Message | Format-List
    } else {
        Write-Host "[$(Get-Date -Format HH:mm)] No alerts" -ForegroundColor Green
    }
}

Check-DCEvents
```

Event ID	Meaning	Severity
4720	New user account created	HIGH — verify immediately
4722	User account enabled	MEDIUM — check if you enabled it
4723	Password change attempt	MEDIUM — watch for brute force
4724	Password reset attempt	HIGH — should only be admins
4725	User account disabled	HIGH — Red Team locking you out?
4726	User account deleted	CRITICAL — IR immediately
4728	Added to Domain Admins	CRITICAL — IR immediately
4732	Added to local Admins	HIGH — investigate source
4740	Account locked out	MEDIUM — brute force indicator
4756	Added to Universal group	HIGH — check group and member
4625	Failed logon	MEDIUM — 5+ = brute force
4648	Explicit credential logon	HIGH — lateral movement
4672	Special privileges assigned	HIGH — escalation attempt

6.2 — New Computer Object Detection

```
# Catch rogue machine joins (gamingpc scenario from quals)
Get-ADComputer -Filter * -Properties Created |
Where-Object {$_.Created -gt (Get-Date).AddMinutes(-30)} |
```

```

Select Name, Created, DistinguishedName

# If rogue machine found:
# 1. Screenshot for IR
# 2. Disable the computer account
Disable-ADAccount -Identity 'roguepc'
# 3. Remove from domain
Remove-ADComputer -Identity 'roguepc' -Confirm:$false

```

6.3 — GPO Tampering Detection

```

# Check for GPOs modified in last 30 minutes
Get-GPO -All |
  Where-Object {$_.ModificationTime -gt (Get-Date).AddMinutes(-30)} |
  Select DisplayName, ModificationTime, Owner

# Check GPO scripts for injected commands
Get-GPO -All | ForEach-Object {
  $report = Get-GPOReport -Guid $_.Id -ReportType XML -ErrorAction SilentlyContinue
  if ($report -like '*Script*') {
    Write-Host "GPO with scripts: $($_.DisplayName)" -ForegroundColor Yellow
  }
}

```

What If: GPO was modified by Red Team

```

# Check what changed
Get-GPOReport -Name 'ModifiedGPOName' -ReportType HTML `
  -Path C:\gpo_report.html

# Restore from backup if you made one
Restore-GPO -Name 'ModifiedGPOName' -Path C:\GPOBackups

# If no backup - disable the GPO until you can review
Set-GPO -Name 'ModifiedGPOName' -Domain chefops.local |
  Set-GPRegistryValue -Key 'HKLM\Software' -ValueName 'test' `
  -Value 'test' -Type String # forces a known state

```

6.4 — 30-Minute Rotation Checklist

Check	Command	Expected Result
Domain Admins membership	Get-ADGroupMember "Domain Admins"	Only your known admins
New computer objects	Get-ADComputer -Filter * -Properties Created	Nothing new
WMI subscriptions	Get-WMIObject -Namespace root\subscription -Class __EventFilter	Empty
Scheduled tasks	Get-ScheduledTask Where TaskPath -notlike "Microsoft**"	None or known only
Replication health	repadmin /showrepl	No errors
Security events	Check-DCEvents function above	No alerts
Firewall profiles	Get-NetFirewallProfile Select Name,Enabled	All True

Phase 7 — Incident Response from DC02

When Red Team activity is confirmed, your IR report from DC02 is what recovers lost points. Based on quals feedback, the most common failure was technical output without business context. Use this structure.

7.1 — Gather Evidence Before Remediating

```
# ALWAYS screenshot before removing anything
# Capture the event that triggered the alert
Get-WinEvent -FilterHashtable @{
    LogName = 'Security'
    Id =
    StartTime = (Get-Date).AddHours(-1)
} | Select TimeCreated, Id, Message | Format-List | Out-File C:\ir_evidence.txt

# Capture current state of compromised object
# Example: rogue Domain Admin found
Get-ADUser 'rogueadmin' -Properties * | Out-File C:\ir_evidence.txt -Append

# Capture network connections at time of incident
Get-NetTCPConnection -State Established |
    Select LocalAddress, LocalPort, RemoteAddress, RemotePort, OwningProcess |
    Out-File C:\ir_evidence.txt -Append
```

7.2 — IR Report Structure

Section	Content
Executive Summary	Business impact in plain language. What service/data was affected. Written for CISO, not technical team.
Timeline	When was activity first detected, what events confirm it, what actions were taken and when.
Technical Details	Event IDs, account names, IPs involved, commands run by attacker. This goes in Supporting Artifacts.
Root Cause	What misconfiguration or vulnerability allowed this. E.g. ms-DS-MachineAccountQuota = 10.
Remediation Taken	Exactly what you did to remove the threat. Account disabled, rule removed, password reset.
Prevention	What GPO or config change stops this from recurring.

✓ Qvals feedback: IR executive summaries must target C-suite. Avoid technical jargon in the summary section. Put all technical detail in Supporting Artifacts. A good IR recovers the full Red Team point deduction for that specific incident.